Message Passing Algorithms in DS

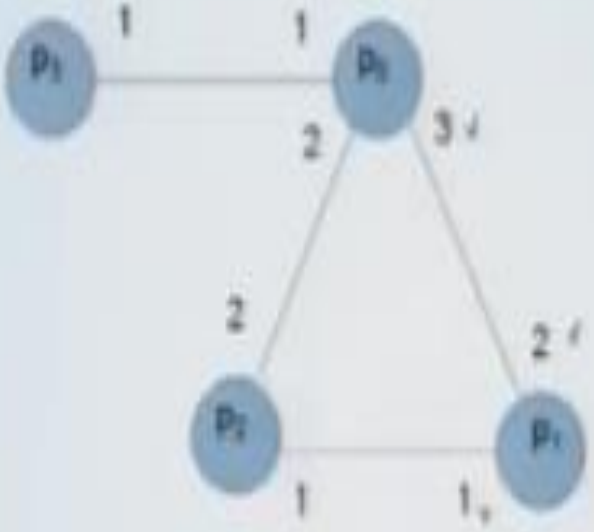
Message-Passing Model

- In a message-passing system, processors communicate by sending messages over communication channels, where each channel provides a bidirectional connection between two specific processors.
- The pattern of connections provided by the channels describes the *topology* of the system.
- The collection of channels is often referred to as the *network*

Message-Passing Model

- More formally, a system or algorithm consists of n processors $p_0, p_1, \ldots, p_{n-1}$; i is the index of processor p_i (nodes of graph)
- bidirectional point-to-point channels (undirected edges of graph)
- each processor labels its incident channels 1, 2, 3,... ; might not know who is at other end

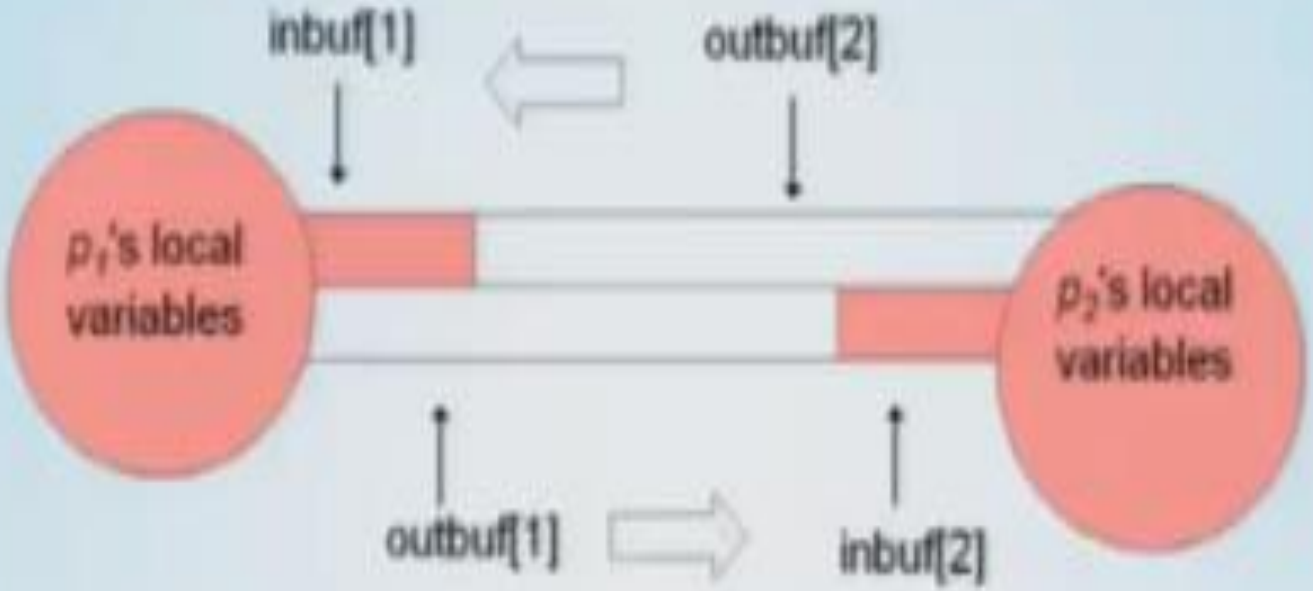
Message-Passing Model



Modeling Processors and Channels

- Processor is a state machine including
  - local state of the processor
  - mechanisms for modeling channels
- Channel directed from processor p_i to processor p_j is modeled in two pieces:
  - *outbuf* variable of p_i and
  - *inbuf* variable of p_j
- *Outbuf* corresponds to physical channel, *inbuf* to incoming message queue

Modeling Processors and Channels



Pink area (local vars + inbuf) is *accessible* state for a processor.

Configuration

- Vector of processor states (including outbufs, i.e., channels), one per processor, is a *configuration* of the system
- Captures current snapshot of entire system: accessible processor states (local vars + incoming msg queues) as well as communication channels.

Events

- Occurrences that can take place in a system are modeled as events.

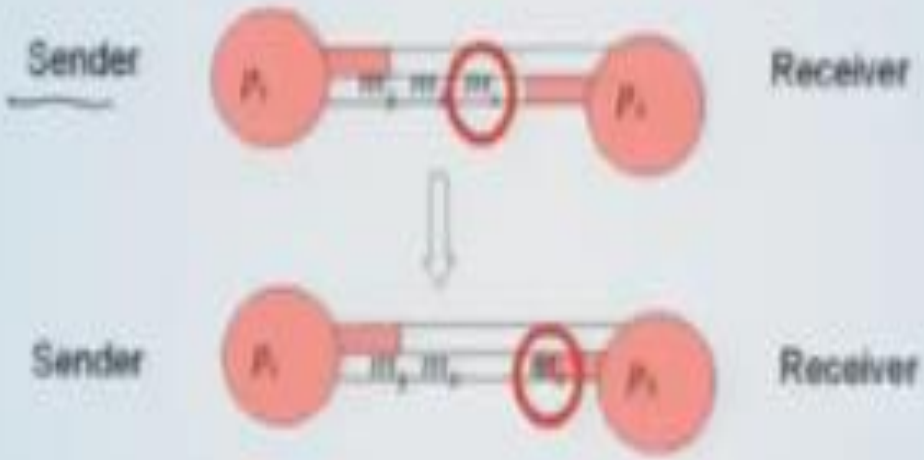
- For message-passing systems, we consider two kind of events:

(i) Deliver event and

(ii) Computation event

(i) Deliver Event

- Moves a message from sender's outbuf to receiver's inbuf; message will be available next time receiver takes a step



(ii) Computation Event

- Occurs at one processor
- Start with old accessible state (local vars + incoming messages)
- Apply transition function of processor's state machine; handles all incoming messages
- End with new accessible state with empty inbufs, and new outgoing messages

Computation Event

There are 3 things happening in computational event:
1) Start with old accessible state
2) Apply transition function of processor's state machine; handles all incoming messages
3) End with new accessible state with empty inbufs, and new outgoing messages

pink indicates accessible state: local vars and incoming msgs
white indicates outgoing msg buffers

Execution

- Format is

  config, event, config, event, config, ...

- in first config: each processor is in initial state and all inbufs are empty
- for each consecutive (config, event, config), new config is same as old config except:
  - if delivery event: specified msg is transferred from sender's outbuf to receiver's inbuf
  - if computation event: specified processor's state (including outbufs) change according to transition function

Types of message-passing systems

There are two types of message-passing systems, asynchronous and synchronous.

1) **Asynchronous Systems:** A system is said to be asynchronous if there is no fixed upper bound on how long it takes for a message to be delivered or how much time elapses between consecutive steps of a processor. An example of an asynchronous system is the Internet, where message (for instance E-mail) can take days to arrive, although often they only take seconds.

2) **Synchronous Systems:** In the synchronous model processor execute in lockstep: The execution is partitioned into rounds, and in each round, every processor can send message to each neighbour, the messages are delivered, and every processor computes based on the messages just received.

1. Asynchronous Message Passing Systems

- An execution is *admissible for the asynchronous model* if
  - every message in an outbuf is eventually delivered
  - every processor takes an infinite number of steps
- No constraints on when these events take place: arbitrary message delays and relative processor speeds are not ruled out
- Models reliable system (no message is lost and no processor stops working)

2. Synchronous Message Passing Systems

- The new definition of admissible captures lockstep unison feature of synchronous model.
- This definition also implies
  - every message sent is delivered
  - every processor takes an infinite number of steps.
- Time is measured as number of rounds until termination.

Broadcast Over a Rooted Spanning Tree

- Broadcast is used to send the information to all.
- Suppose processors already have information about a rooted spanning tree of the communication topology
  - tree: connected graph with no cycles
  - spanning tree: contains all processors
  - rooted: there is a unique root node
- Implemented via *parent* and *children* local variables at each processor
  - indicate which incident channels lead to parent and children in the rooted spanning tree

Broadcast Over a Rooted Spanning Tree: Concept

- root initially sends *M* to its children
- when a processor receives *M* from its parent
  - sends *M* to its children
  - terminates (sets a local boolean to true)

Broadcast Over a Rooted Spanning Tree: Algorithm 1

Algorithm 1 Spanning tree broadcast algorithm.

Initially $\langle M \rangle$ is in transit from p_r to all its children in the spanning tree.

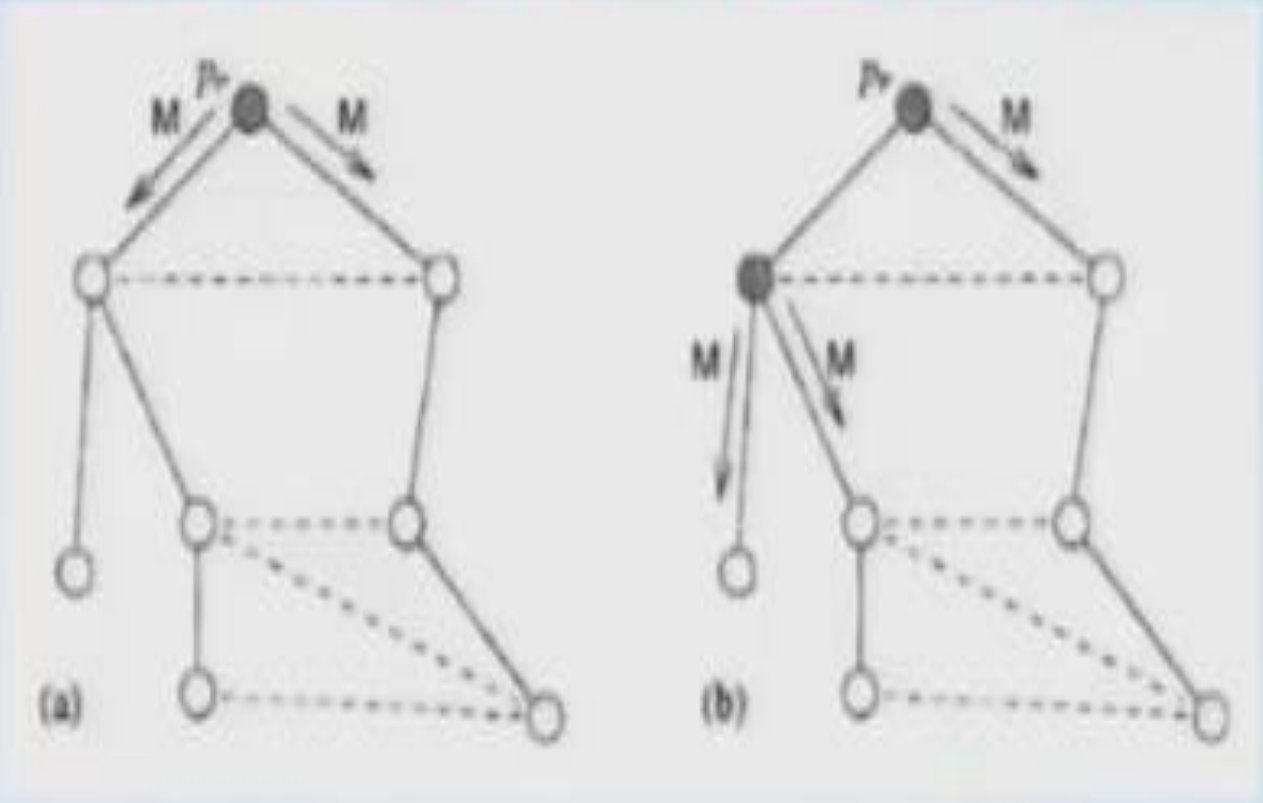
Code for p_r:

1: upon receiving no message: // first computation event by p_r
2: terminate

Code for p_i, $0 \leq i \leq n-1, i \neq r$:

3: upon receiving $\langle M \rangle$ from parent:
4: send $\langle M \rangle$ to all children
5: terminate

Broadcast Over a Rooted Spanning Tree: Example



Two steps in an execution of the broadcast algorithm

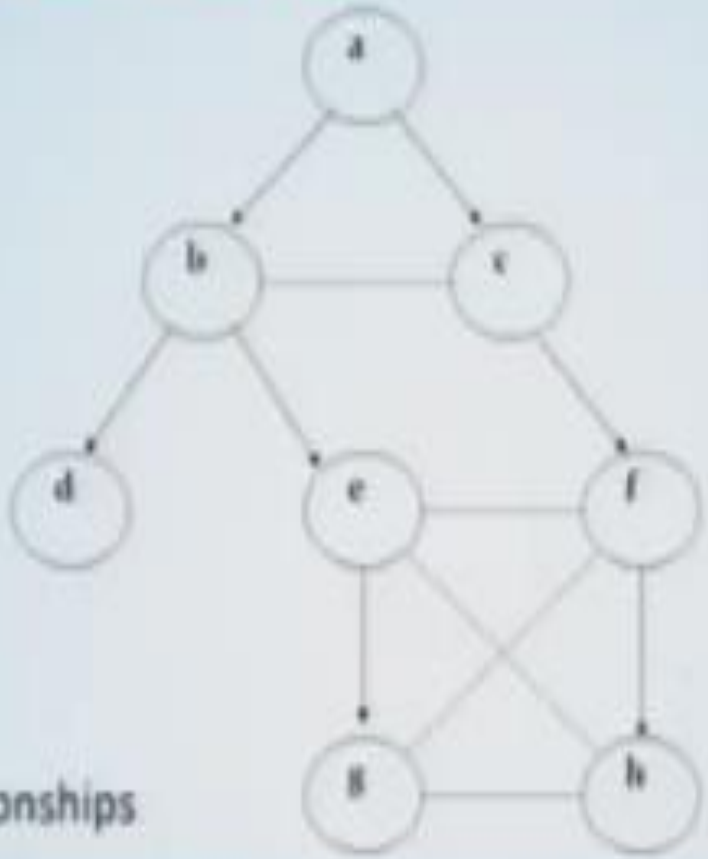
Complexity Analysis

- Synchronous model:
  - time is depth ***d*** of the spanning tree. (at most *n – 1 when chain)*
  - number of messages is ***n - 1***, since one message is sent over each spanning tree edge
- Asynchronous model:
  - same as synchronous ie time (d) and messages (n-1)

Convergecast: Concept

- Convergecast is used to collect the information.
- Again, suppose a rooted spanning tree has already been computed by the processors
  - *parent* and *children* variables at each processor
- Do the opposite of broadcast:
  - leaves send messages to their parents
  - non-leaves wait to get message from each child, then send combined (aggregate) info to parent

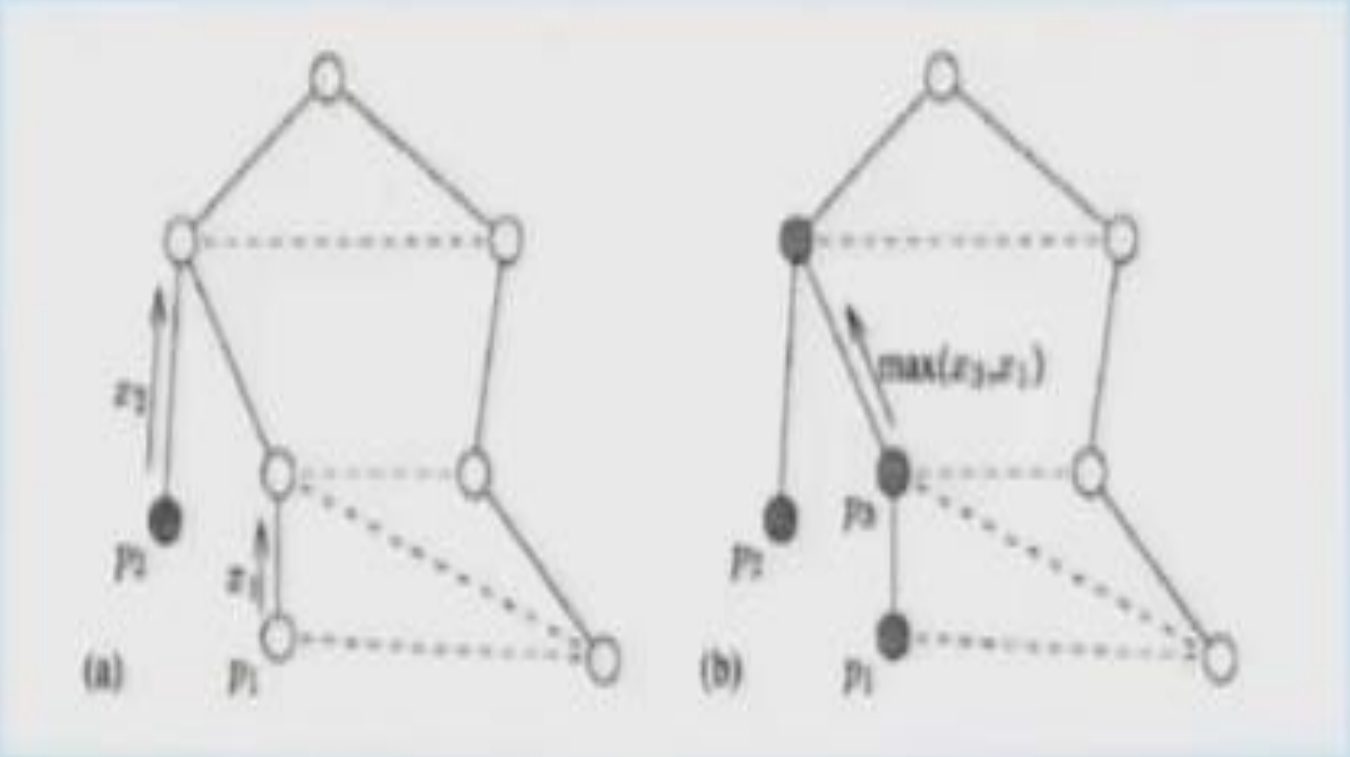
Convergecast: Example



dotted lines:
non-tree edges

solid arrows:
parent-child relationships

Convergecast: Example



Two steps in an execution of the convergecast algorithm

Finding a Spanning Tree Given a Root

- Having a spanning tree is very convenient.
- How do you get one?
- Suppose a distinguished processor is known, to serve as the root.

Finding a Spanning Tree Given a Root

- root sends *M* to all its neighbors
- when non-root *first* gets *M*
  - set the sender as its parent
  - send "parent" msg to sender
  - send *M* to all other neighbors (if no other neighbors, then terminate)
- when get *M* otherwise
  - send "reject" msg to sender
- use "parent" and "reject" msgs to set *children* variables and know when to terminate (after hearing from all neighbors)

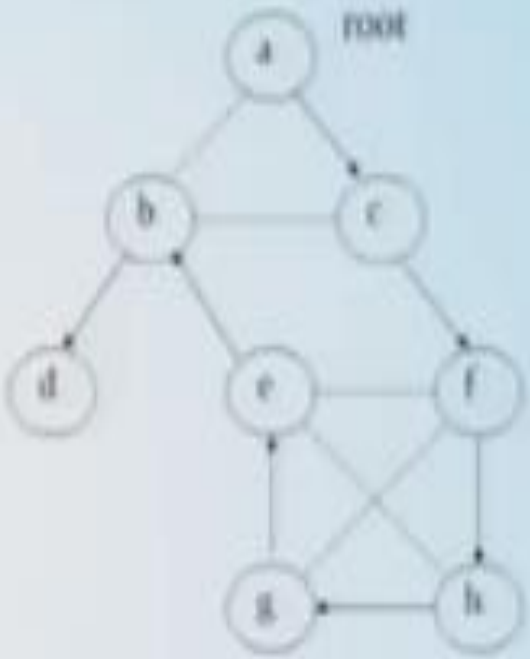
Algorithm 2 Modified flooding algorithm to construct a spanning tree: code for processor p_i, $0 \leq i \leq n-1$.

Initially *parent* $= \perp$, *children* $= \emptyset$, and *other* $= \emptyset$.

```
1:  upon receiving no message:
2:      if p_i = p_r and parent = ⊥ then          // root has not yet sent ⟨M⟩
3:          send ⟨M⟩ to all neighbors
4:          parent := p_i

5:  upon receiving ⟨M⟩ from neighbor p_j:
6:      if parent = ⊥ then                        // p_i has not received ⟨M⟩ before
7:          parent := p_j
8:          send ⟨parent⟩ to p_j
9:          send ⟨M⟩ to all neighbors except p_j
10:     else send ⟨already⟩ to p_j

11: upon receiving ⟨parent⟩ from neighbor p_j:
12:     add p_j to children
13:     if children ∪ other contains all neighbors except parent then
14:         terminate

15: upon receiving ⟨already⟩ from neighbor p_j:
16:     add p_j to other
17:     if children ∪ other contains all neighbors except parent then
18:         terminate
```

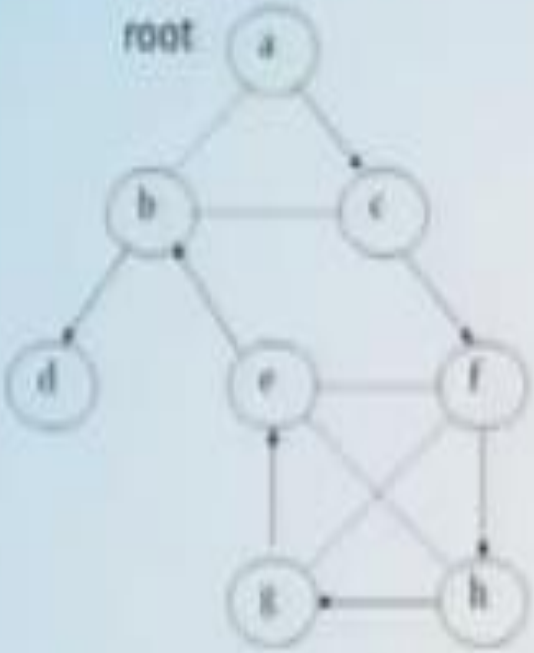
Execution of Spanning Tree Algorithm

Synchronous: always gives breadth-first search (BFS) tree



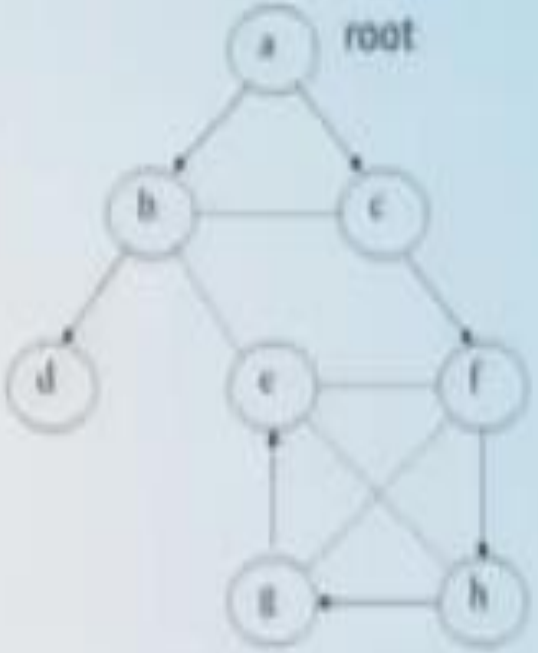
Asynchronous: not necessarily BFS tree

Execution of Spanning Tree Algorithm



An asynchronous execution ✓ gave this depth-first search (DFS) tree. Is DFS property guaranteed?

No!



Another asynchronous execution results in this tree: neither BFS nor DFS

Finding a DFS Spanning Tree Given a Root

- when root first takes step or non-root first receives *M*:
  - mark sender as *parent* (if not root)
  - for each neighbor in series
    - send *M* to it
    - wait to get "parent" or "reject" msg in reply
  - send "parent" msg to *parent* neighbor
- when processor receives *M* otherwise
  - send "reject" to sender
  - use "parent" and "reject" msgs to set *children* variables and know when to terminate

Algorithm 3 Depth-first search spanning tree algorithm for a specified root; code for processor p_i, $0 \leq i \leq n-1$.

Initially *parent* $= \perp$, *children* $= \emptyset$, *unexplored* $=$ all neighbors of p_i

```
1:  upon receiving no message:
2:      if p_i = p_0 and parent = ⊥ then                    // root wakes up
3:      parent := p_i
4:      explore()

5:  upon receiving ⟨M⟩ from p_j:
6:      if parent = ⊥ then                                  // p_i has not received ⟨M⟩ before
8:          parent := p_j
9:          remove p_j from unexplored
10:         explore()
11:     else
12:         send ⟨already⟩ to p_j                           // already in tree
13:         remove p_j from unexplored
14: upon receiving ⟨already⟩ from p_j:
15:     explore()

16: upon receiving ⟨parent⟩ from p_j:
17:     add p_j to children
18:     explore()
19: procedure explore():
20:     if unexplored ≠ ∅ then
21:         let p_k be a processor in unexplored
22:         remove p_k from unexplored
23:         send ⟨M⟩ to p_k
24:     else
25:         if parent ≠ p_i then send ⟨parent⟩ to parent
26:         terminate                                       // DFS subtree rooted at p_i has been built
```

Finding a DFS Spanning Tree Given a Root

- Previous algorithm ensures that the spanning tree is always a DFS tree.
- Analogous to sequential DFS algorithm.
- Message complexity: $O(m)$ since a constant number of messages are sent over each edge
- Time complexity: $O(m)$ since each edge is explored in series.

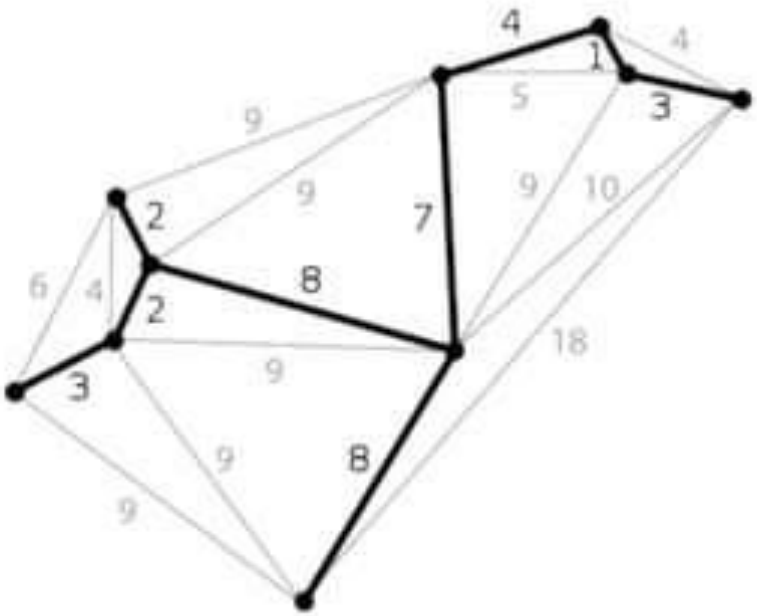
Finding a Spanning Tree Without a Root

- Assume the processors have unique identifiers (otherwise impossible!)
- *Idea:*
  - each processor starts running a copy of the DFS spanning tree algorithm, with itself as root
  - tag each message with initiator's id to differentiate
  - when copies "collide", copy with larger id wins.
- Message complexity: $O(nm)$
- Time complexity: $O(m)$

Minimum spanning trees

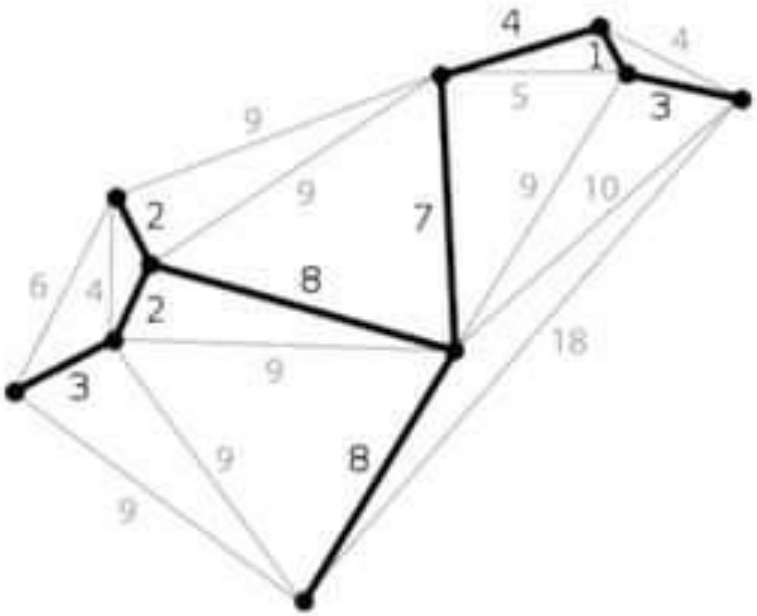
Ref: Wiki

- Definition (in an undirected graph):
  - A spanning tree that has the smallest possible total weight of edges



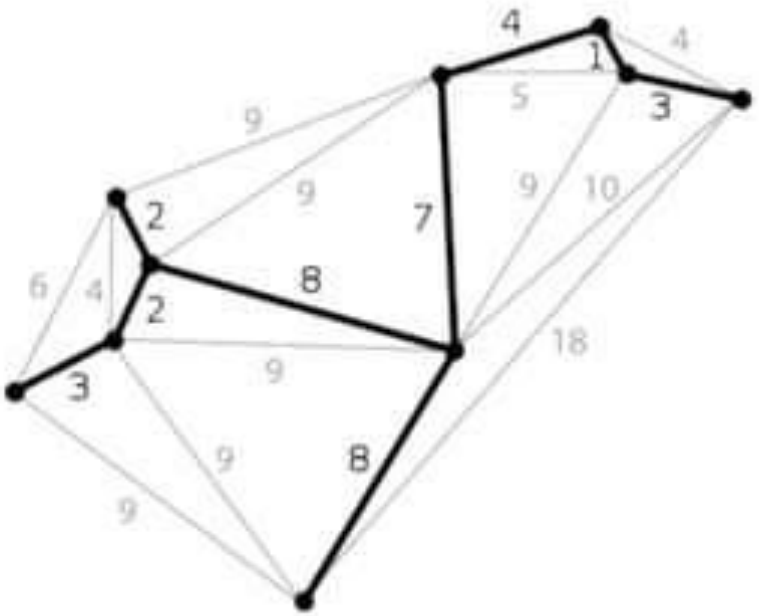
Minimum spanning trees

- Useful in broadcast:
  - Using a flood on the MST has the smallest possible cost on the network



Minimum spanning trees

- Useful in point to point routing:
  - Minimizes the max weight on the path between any two nodes



Distributed Spanning Tree

GHS Distributed MST Algorithm

Ref: NL

- By Gallagher, Humblet and Spira
- Each node knows its own edges and weights

GHS Distributed MST Algorithm

- Works in levels
- In level 0 each node is its own tree
- Each tree has a leader (leader id == tree id)
- At each level k:
  - All Leaders execute a convergecast to find the min weight boundary edge in its tree
  - It then broadcasts this in its tree so that the node that has the edge knows
  - This node informs the node on the other side, which informs its own leader

GHS Distributed MST Algorithm

- Observation 1:
  - We are possibly merging more than two trees at the same time
  - Problem: who is the leader of the new tree?
- Observation 2:
  - The merged tree is a tree of trees: it cannot have a cycle
  - We can assign a direction to each edge and each node (tree) has an outgoing edge
  - There must be a pair of nodes (trees) that select each-other (otherwise the merged tree is infinite)
  - We select the edge used to merge these two trees
    - Select the node with higher ID to be leader
  - The leader then broadcasts a message updating leader id at all nodes.

GHS Distributed MST Algorithm

- Complexity:
- The number of nodes at each level k tree is at least 2^k
- Since starting at size 1, the number of nodes in the smallest tree at least doubles every level
- Therefore, there are at most O(log n) levels

GHS Distributed MST Algorithm

- Complexity:
- At each level, at each tree, we use constant number of broadcasts and convergecasts
- Each level costs O(n) time
- Total costs : O(n log n) time

GHS Distributed MST Algorithm

- Complexity:
- At each level, at each tree, we use constant number of broadcasts and convergecasts
- Each level costs O(n) messages
- Total costs : O(n log n + |E|) messages